DOCUMENTATION

DOCUMENTATION — DIVERGENCE

The "Documentation" file required by the ML Prompt Engineering track's submission rules: *"a detailed documentation that explains the reasoning behind each node in the AI workflow, how it works and any other necessary data."*

This file is the single answer to that requirement. It is organized exactly as asked — per node: why it exists, how it actually works, and the data it needs — plus the model registry the track separately asks the flowchart to name. Where a claim below benefits from more depth than fits here, it links to the specific doc that has it, rather than duplicating pages of content.

---

1. What the workflow is for, in one paragraph

A resident freelancer in India is paid 5,000 USDC by a foreign client. A single unstructured prompt — or a Koinly-style tool — prints one number for that receipt: the SBI TT buying rate next published after the payment, **₹4,69,750**. A different method, resting on the identical statute, not a lesser one — the domestic market's high print for the day, through the retrieved USDC/USDT peg — prints **₹5,17,618.76**. The difference is **₹47,868.76**, about 10.19% of the payment, and nothing in the market discloses that it exists: every commercial tool checked for this project resolves the identical missing-price fact pattern differently and silently, with no flag that a choice was made (`prior-art/OBJ-1.md`). This workflow reads the actual law text instead, computes all twelve defensible valuations mechanically, discloses the ₹47,868.76 spread and what drives it (§3, node ⚙B), and still leaves the filer with one elected figure — theirs, with a recorded basis (§3, node ⚙D) — instead of a guess dressed as a fact or a hallucinated citation that sounds right.

This is a decision aid, not an oracle. The patient-decision-aid literature (IPDAS Evidence Update 2.0, *Medical Decision Making*, 2021) and selective prediction / reject-option classifiers (SelectiveNet, Geifman & El-Yaniv, arXiv:1901.09192) are the two existing fields where declining to collapse to one answer is the contribution being measured, not a failure mode. This workflow's twelve-figure lattice and its `lacuna/insufficient_evidence` certainty labels belong to that family, not to a forecasting tool graded on always answering — which is also why every uncertain figure here is stated as a number (a spread, a driver, a percentage) rather than a verbal hedge like *"may vary"* or *"depending on interpretation"*: van der Bles et al. (PNAS, 2020) find numeric ranges cost little public trust while vague verbal hedging reads as less trustworthy than a precise number (direction only — this project has not independently re-verified the paper's own effect sizes). Full framing: the root `README.md`.

2. The model registry — which LLM is used where, and why

All models are served by **Featherless** (`https://api.featherless.ai/v1`, OpenAI-compatible), never a frontier API, and never silently substituted — `llm_call.py`'s `provider_name()` refuses to fall back to another provider (decision D44), so a result row can never quietly come from a different model than the one named for it.

Slot	Real model	Used by	Why this one
`small`	`Qwen/Qwen2.5-7B-Instruct`	Node 1 (Extract), Node 2 (Gap detector)	These two tasks are extraction and enumeration against a short context — a 7B model is enough, and using it here rather than the 72B model everywhere is itself part of the cost/sustainability story (see `cost-model-output.txt`)
`large`	`Qwen/Qwen2.5-72B-Instruct`	Node 3 (Income tax resolver), Node 4 (GST resolver)	These two read long verbatim statutory text and have to reason about scope, exceptions, and certainty — the two hardest reasoning tasks in the pipeline

Slot	Real model	Used by	Why this one
`adversarial`	`mistralai/Mistral-Large-Instruct-2411`	Node 5 (Adversarial checker)	**Deliberately a different model family from the resolvers** (decision D41). A model checking its own reasoning has a documented self-consistency bias — it tends to agree with itself. Grounding the critique in a different model, plus mechanical inputs (the citation matcher's verdicts, the gap list) it did not produce, is the actual point of this node, not incidental. `check_llm.py` warns at runtime if this rule is ever accidentally broken.

`check_llm.py` confirms all three slots resolve to real, working model IDs on the current account before any real run starts — a few hundred tokens, run first every session, specifically so a broken model slot is caught in ten seconds rather than mid-evaluation.

3. Every node — reasoning, mechanics, data

Ten steps: five make an LLM call (🤖, can be wrong) and five are plain Python (⚙, cannot invent anything — an `if` statement, a string match, arithmetic, a template). The full visual version of this table, with the required human-input markers, is `flowchart.png`. *Update, 21 Aug: was nine/four until ⚙ E (`scope_enforcer.py`) was added — `DECISION-D59.md`.*

🤖 1 — EXTRACT (`node1_extract.py`)

Reasoning. Nothing downstream can be trusted if the facts it starts from are wrong or unverifiable, so the very first step has to make its own output checkable, not just plausible-sounding.

How it works. Takes the raw human-provided document (typed text, a PDF, or a photo). Sends it with prompt `step22drop/prompts/01-extract.md` to the `small` model. The prompt gives a fixed 13-field name contract (the union of every field any of this project's six ground-truth cases actually needs), so the model can't invent its own field names that a scorer would then mark wrong for using a synonym. Every returned field carries a `confidence` level and a `source_span` — the literal substring of the input the value came from — so a human can check the extraction against the document in seconds rather than trusting it.

Data needed. The raw document only. No corpus, no prior state.

Security, added 22 Aug (D62). This is the one node that reads untrusted, user-supplied text and hands it to a model. `injection_scanner.py` scans the raw input for known injection phrasings before it's sent (and the model's own output afterward), advisory not blocking. The document text itself is wrapped in a fresh, random per-call nonce marker (`<<<DOCUMENT-{nonce}-START/END>>>`), with an explicit system-prompt instruction that text between those markers is data, never instructions. Both verified offline against a constructed adversarial case (`cases/ADV1-injection/`); full account, including what this does not guarantee, in `SECURITY.md` and `DECISION-D62.md`.

🤖 2 — GAP DETECTOR (`node2_gaps.py`)

Reasoning. This step runs **before** anything reasons about a conclusion, on purpose. A gap discovered as an afterthought reads like an excuse; a gap established as a fact up front becomes a hard constraint the rest of the pipeline cannot reason its way around.

How it works. Takes `facts{}` from Node 1 plus a small, fixed set of evidence-requirement excerpts (what a FIRC is, what a purpose code is, what GST needs to prove an export). Prompt `02-gap-detector.md`, `small` model. Returns `missing[]`, each entry naming what's absent, why, what regimes it blocks, and whether it's `obtainable` at all — a `not_for_this_route` item (like a bank certificate for a wallet-to-wallet transfer) is a structurally different kind of gap from one the user simply forgot to attach.

Data needed. `facts{}` and the evidence-requirement excerpts only — deliberately not the full corpus, so this step cannot pre-empt a legal conclusion it isn't supposed to be making yet.

⚙ A — GAP CONSTRAINT ENFORCER (`gap_enforcer.py`) — no model

Reasoning. Telling a model "respect the gap list" is a request. A model that sounds confident can talk its way past a request. Code cannot be talked out of an `if` statement.

How it works. Scans every regime conclusion for a non-empty `depends_on_missing[]`. If present, that conclusion's `certainty` is overwritten to `insufficient_evidence` — unconditionally, regardless of how the model worded its own confidence.

Data needed. `missing[]` (Node 2) and every regime conclusion produced so far.

⚙ B — VALUATION LATTICE (`node3_valuation.py`) — no model, no API

Reasoning. The headline rupee figure is this project's single highest- stakes output. It must never be a token prediction (decision D38) — a number a human typed by hand is not meaningfully better than one a model guessed; both are unverifiable claims dressed as facts.

How it works. Reads a case's real sourced inputs — SBI's published telegraphic-transfer rates (archived PDFs, cross-checked against a live public GitHub mirror before trusting a new figure), and, where a crypto leg exists, a daily market candle plus a stablecoin/dollar peg reading. Enumerates every combination of official date × market reading × currency peg, and computes each resulting rupee figure by plain arithmetic. Where there is a genuine dispute (two official dates, no rate published on the actual settlement date) it produces up to twelve figures; where a receipt has no real valuation dispute at all (a plain domestic invoice, or an ordinary weekday bank wire with a normally-published rate) it correctly produces exactly one, with zero spread — the schema (`minItems: 1`, decision D51) allows both, and treats the difference as data, not error.

Data needed. A case's `canonical_case.json`/`--case` input: officially published rates with source URLs and retrieval dates, and (for crypto cases) a market data candle. Never a guessed number.

🤖 3 / 🤖 4 — INCOME TAX & GST RESOLVERS (`node_resolver.py`, prompts `03-income-tax.md` / `04-gst.md`)

Reasoning. This is where the actual legal reasoning happens, and where this project's central finding was measured: a model given verbatim statute and a genuinely underdetermined question will confidently reach for the nearest rule that mentions the right words, even when that rule's own scope excludes the facts. Five confirmed instances of this exact pattern exist in this project's own resolver output — see §5 below.

How it works. Each resolver is given **only** the statutory text for its own regime — income tax never sees GST text and vice versa, not because the prompt asks it not to cite the other regime, but because that text is structurally never in its context window (decision C22). Both use the `large` model. Each conclusion must carry exactly one load-bearing citation (not several joined into one string — found live, decision D46, a five-citation string only ever got its first reference actually checked) and a certainty label: `settled | inference | open_texture | lacuna | contested | insufficient_evidence`. `lacuna` — the strongest, easiest to get wrong claim in the whole schema — means a provision demands a method and none in the given text supplies one; it is not the same claim as `insufficient_evidence` (missing facts, not a missing rule).

Why this is a different kind of indeterminacy than the ML literature's nearest named concept, checked rather than assumed. Guerdan et al. (NeurIPS 2025, arXiv:2503.05965) name "rating indeterminacy" — many rating tasks admit multiple defensible answers, and forcing a single label anyway before validating against it selects a judge system up to 31% worse than keeping the full defensible set. Real, measured, and the honest competitor to `contested/open_texture/lacuna` — but a different failure to fix. Their indeterminacy lives in the rating rubric: a better-written question resolves it. This project's lives in the statute itself: no better-written prompt closes a gap Parliament left, which is the entire reason these three certainty values are enforced as a closed enum rather than left to free text. Full card: `prior-art/READING-CARDS.md`, #8. The prompt contains an explicit **SCOPE GATE** instruction, generalized after three separate real failures of the identical shape (Rule 57's own column B, Rule 206's own "foreign currency" opening words, Rule 243's own reporting-obligations scope) all misapplied a real, current, correctly- quoted provision outside where its own text says it reaches.

Data needed. `corpus/verbatim/` files scoped to the regime (income tax: 10 files; GST: 3 files — see the header of each prompt file for the exact list), plus `facts{}`, `missing[]`, and Node ⚙B's valuation figures (never recomputed by the model, only cited).

⚙ C — CITATION MATCHER (`citation_matcher.py`) — no model

Reasoning. A citation that sounds right and a citation that is right are indistinguishable to a reader unless something actually checks it. This is the check.

How it works. String-matches every citation a resolver produced against the real corpus text in `corpus/tier-a/`, and separately checks it is current for the **stated tax year** — both numbering systems (the 1961 Act and the 2025 Act) are simultaneously live depending on which tax year a record is about, and a citation given with no tax year is rejected outright, unconditionally. If a citation fails either check, the entire conclusion resting on it is **dropped**, not flagged. This has caught five of this project's own historical citation errors automatically, including one where two retired corpus files were silently shadowing their replacements for four months (`ITERATION-STORY.md` item 2).

Data needed. `corpus/tier-a/` (17 files, each carrying its own current citation, former citation, and the tax year it governs) and the tax year stated on the record.

Why a frozen verbatim corpus rather than embedding retrieval — now with an external number attached, not just an internal preference. Cymbler, Guez and Fabre, "*Temporal Misgrounding in Legal RAG*" (arXiv:2608.09393, independently verified) built a 32,436-article-version, 93-year French tax code benchmark and found static RAG retrieves the date-applicable version of a statute **0% of the time** — 2.7% accuracy overall, against 98.3% for a purpose-built multi-version retriever. This project's corpus is frozen and verbatim, with an explicit tax-year currency check, precisely because that is the failure mode a naive retrieval approach over statutory text predictably hits. Full card: `prior-art/READING-CARDS.md`, #7.

⚙ E — SCOPE-REACH ENFORCER (`scope_enforcer.py`) — no model

Added 21 Aug, `DECISION-D59.md`.

Reasoning. ⚙ C proves a citation is real and current. It does not, and cannot, prove the citation actually reaches the facts it was applied to — its own `LIMITATIONS` section says so directly. This project has three real, hand-verified instances of exactly that gap in its own resolver history (§5 below), each previously caught only when Node 5 happened to run and happened to land the attack. This turns those three, specifically, into a guarantee.

How it works. For each kept conclusion, matches its citation to a corpus `provision_id` (the same ref-extraction ⚙ C already trusts) and, if that provision is one of the three this project has proven does not reach a virtual-digital-asset

receipt, drops the conclusion — unless its `certainty` is `lacuna`, in which case the citation is being used to prove the absence, not claim the authority, and is left alone. Tested against a real regression before shipping: a version without that exemption dropped this project's own frozen demo record's correct "no rule found" finding — see `DECISION-D59.md` for the full account.

Data needed. The record's `regimes[]` (after ⚙ C) and `facts{}` — specifically the `asset` field, to tell a virtual-digital-asset receipt apart from a genuine foreign-currency one.

Stated limitation. Three provisions, not a scope-reading model. A fourth misapplied provision this project has never analysed is exactly as invisible to this file as it was before. `s.393(1)`'s own scope-reach bug (`DECISION-D55.md`) was deliberately left out — telling its correct use apart from its historical misuse needs the outcome's polarity, not just citation and facts, and stays Node 5's job.

🤖 5 — ADVERSARIAL CHECKER (`node5_adversarial.py`, prompt `05-adversarial.md`)

Reasoning. Every other check in this pipeline verifies a citation is real and current. None of them can tell whether a real, current, correctly-quoted citation actually reaches the facts it was applied to. This node exists specifically to attack that gap.

How it works. Given every kept conclusion, the full gap list, the valuation lattice, and the **entire unscoped corpus** (not scoped like Nodes 3/4 — its whole job is cross-checking scope, so it needs everything), the `adversarial` model is asked to attack each conclusion and say whether its attack landed, publishing the result either way — `attacked[]` is never used to silently improve the answer, only to critique it on the record. A deterministic guard (`_reject_upward_revisions`) then checks each proposed downgrade by matching the attack's free-text target back to the regime it quotes (word-overlap, not substring — the model paraphrases) and rejects any "downgrade" that isn't actually less certain than the conclusion already was, recording the rejection rather than letting a nonsensical label pass silently.

Data needed. Every conclusion produced so far, `missing[]`, the valuation lattice, and the full corpus, unscoped.

⚙ D — DISCLOSURE COMPOSER (`node7_disclosure.py`) — no model

Reasoning. *"A document whose purpose is to be trustworthy cannot be produced by something that can hallucinate"* — the page a reader opens first has to be assembled the same way the headline number is: by code that cannot invent anything, not a model that could.

How it works. A deterministic HTML template. Section order is fixed in code and is itself part of the argument: absence first, the valuation range second, a single confident answer never (unless the record itself has no real dispute, in which case it says so plainly), what was cited fourth, and what was attacked fifth. Real, structural accessibility markup (heading hierarchy, a `<main>` landmark, `aria-labelledby`/`aria-label` on every section and the one purely visual element) is built into the template itself, so every record it renders gets it, not just a demo.

Data needed. One complete schema-valid record — nothing else; this step makes no model call and needs no live data.

---

4. Human input — where it is actually necessary

The workflow needs exactly one thing from a human to run at all: **the invoice and payment record** at the very start (Node 1's input). Nothing else in the automated chain requires a human decision to proceed — nodes 2 through ⚙D run without intervention once the input document is provided.

One further, explicitly **optional** human-input point exists in the output itself: the taxpayer may tick which of the disclosed figures they end up filing. No option is pre-selected (decision from `scope.md` Part 8: *"a default is a recommendation,"* and this workflow refuses to make tax decisions for anyone) — the record is already complete and valid with nothing ticked. Both points are marked explicitly, in purple, on `flowchart.png`.

5. The one finding this workflow produced that it wasn't built to look for

A model given verbatim statute and a genuinely underdetermined question will reliably reach for the nearest rule that mentions the right words and apply it with confidence. This happened, confirmed, **five separate times** in this project's own resolver output — Rule 57 row 7 (scoped to a different section by its own column B), Rule 206 row 3 (scoped to foreign currency, which a virtual digital asset is

defined not to be), Rule 243(8)(e) (scoped to a reporting service provider, not a taxpayer), and s.393(1) twice (inverted who the section addresses; then a foreign-payer exemption the section's own text does not state). Three of the five were caught by Node 5 on data nobody planted — the node built specifically to catch this class of error, catching it on itself. **None of the five were visible to any of this project's five accuracy metrics.** Full account, each instance verified against the actual gazette text before being written down: `DECISION-D50.md`, `DECISION-D54.md`, `DECISION-D55.md`.

The failure has a published name, found independently and re-verified rather than assumed: Silent Scope Omission (SSO). Chen, Li, Wan and Yuan, *"From Statute to Control Flow: Span-Grounded Deontic Trees for Defeasible Scope Parsing"* (KDD '26; arXiv:2606.08932) define it as a model applying a general rule while silently dropping a nested exception, producing output that *looks* compliant but breaks exactly where the exception exists. Their diagnosis of the mechanism — an *"Auditability Trap"*: models retrieve the relevant span but fail to attach it to its correct logical parent, so finding the rule outperforms understanding its scope — is a precise description of what this project's own five instances are. **Be exact about the fit, not a stretch:** their SSO is exceptions dropped from *inside* a provision; this project's is a provision's own *governing* scope — column B, its opening words, who it addresses. Same family — the citation stays real while the thing that should have limited it goes missing — not the identical failure. Full card: `prior-art/READING-CARDS.md`, #6.

Update, 21 Aug — three of the five no longer depend on Node 5 running at all. `scope_enforcer.py` (⚙ E, `DECISION-D59.md`) encodes the Rule 57, Rule 206/207 and Rule 243/247 findings as deterministic code: a conclusion citing any of the three against a virtual-digital-asset receipt is now dropped unconditionally, the same `accept=False` semantics ⚙ C already uses for a fabricated citation. The two `s.393(1)` instances are deliberately **not** included — that failure turns on which direction a conclusion argues, not on citation + facts alone (the correct current D1 answer legitimately cites the same provision to explain why no obligation arises), and remain Node 5's job alone. This is not a claim that scope- reach is solved in general — a sixth, unanalysed misapplication is exactly as invisible to the code as it always was. It is a claim that these three, specific, already-paid-for findings can no longer reach a record whether or not Node 5 happens to be run that day.

A closer precedent, found later and independently re-verified rather than taken on trust: this failure direction is the one a published, peer-reviewed statutory-reasoning experiment already measured. Blair-Stanek, Holzenberger and Van Durme, *"Can GPT-3 Perform Statutory Reasoning?"* (ICAIL 2023, DOI 10.1145/3594536.3595163; arXiv:2302.06100) tested GPT-3 on synthetic statutes it could not have memorized. Two numbers, each independently confirmed

against the paper's own text (not the abstract alone, which doesn't carry them): GPT-3's best result on the real-statute SARA benchmark, **71% (71/100), against a prior BERT-based state of the art of 59% (59/100)**; and, on the synthetic statutes, of **2,272 total errors, 2,204 were false positives** — the model asserting a rule applies when it does not — against only 61 false negatives. Their errors are on statutes invented for the experiment, so a model reaching for a plausible-sounding rule can never be right by accident. This project's three code-confirmed instances (Rule 206/207, Rule 57, Rule 243/247) are the real-statute counterpart: the same directional failure, on real, current, correctly-quoted provisions that pass every citation-existence check and are still wrong. *(One caution, disclosed rather than smoothed over: a research pass that named this same paper also relayed several more granular sub-figures — a 74%/64% split and a specific title- identification percentage — that did not match this independent re-verification of the paper's own text. Only the two numbers above are used here for that reason; the rest are not asserted.)*

This also sits in the same family as a second published finding, not just an analogy we're drawing. Magesh, Surani, Dahl, Suzgun, Manning and Ho (Stanford RegLab), *"Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools"* (Journal of Empirical Legal Studies, 2025; arXiv:2405.20362) splits legal AI hallucination into two dimensions — correctness and *groundedness* — and names the case where "retrieval results are poor or irrelevant, but the model happens to produce the correct answer, falsely asserting that an unrelated source supports its conclusion" (checked against their own framing, not paraphrased from memory). Our five instances are a specific, narrower variant of the same groundedness failure: the source isn't unrelated or irrelevant — it's real, current, and correctly quoted — but its own scope, read plainly, doesn't reach the facts it's cited for. Worth naming precisely rather than claiming a broader match than the evidence supports: we are not asserting this project rediscovered Magesh et al.'s exact taxonomy category, only that both are instances of a model treating citation validity as a proxy for citation relevance, measured independently, in two different legal systems. Six more papers grounding specific design decisions in this project (why the adversarial checker is a different model family, why abstention isn't the right frame, why temperature-0 instability isn't unique to this pipeline, why a frozen verbatim corpus beats embedding retrieval for statutory text, why this project's indeterminacy is not the same as measurement-rubric indeterminacy), each checked independently rather than assumed from a title: `prior-art/READING-CARDS.md`.

6. Where the deeper material lives

- **All 34 decision documents, the mental model, and the full run guide, merged into one chronological file:** `START-HERE.md`

- Full node-by-node rationale with predicted failure rates, written before any run: `architecture.md`
- Plain-language run-through of an actual invocation: `PIPELINE-FLOW.md`
- Every real metric, all six cases, all three arms, including where the workflow loses: `results.md`
- The single-prompt-vs-workflow comparison the track separately requires: `SAMPLES.md`
- Seven curated moments of what broke and what changed: `ITERATION-STORY.md`
- Real, verified published research grounding specific decisions, each checked against its own abstract before being cited: `prior-art/READING-CARDS.md`
- Every individually dated design decision: `DECISION-D*.md`, up to `DECISION-D74.md`